

CISC3024 Pattern Recognition

AI Assignment #1 — Deep CNN / Deep Autoencoder

ConvNeXt V2: Co-designing a ConvNet with a Fully Convolutional Masked Autoencoder (CVPR 2023), reproduced on CIFAR-10

Name: HOI KAI CHENG Student ID: UC325381

AI tool used: Cursor IDE in agent mode (model: Claude). As required by the assignment, every line of code, every experiment and this report were produced by the AI agent; my own contribution was limited to writing prompts and making the choices recorded in Section 1.

1. How I asked the AI tool to find the algorithm

I used the agent mode of the Cursor IDE, which can read files, run shell commands, write code and search the web, so a single conversation could carry the whole assignment from literature search to report. My first message simply attached the assignment PDF and the list of required report sections:

```
c:\Users\th255\Downloads\AIassign1.pdf
Your AI assignment report in Word/PDF format should contain the following details
(1) how you ask AI tools to find the algorithm, (2) algorithm description,
(3) how AI implements the algorithm, (4) experiment settings and results,
(5) what you have learnt from this AI assignment, and (6) a webpage link of your
source codes.
then save all the file into a new folder 'hw2'
```

The agent read the PDF, extracted the constraints ("a recent algorithm of Deep CNN or Deep Autoencoder with an application in computer vision and pattern recognition", "you are NOT allowed to write any code by yourself"), inspected my machine (Python 3.12, PyTorch 2.13, 20 CPU cores, no GPU, GitHub CLI already logged in) and then, instead of guessing, came back with three multiple-choice questions. I quote them because they are effectively the prompt that selected the algorithm:

Question asked by the agent	Options offered	My choice
Which recent algorithm should the assignment cover? (must be a recent Deep CNN or Deep Autoencoder for computer vision)	ConvNeXt V2 (CVPR 2023, FCMAE + GRN; recommended) / ConvNeXt V1 (CVPR 2022) / convolutional masked autoencoder for anomaly detection / EfficientNetV2-RepVGG	ConvNeXt V2
Section (6) needs a webpage link to the source code. How should I publish it?	public GitHub repo (recommended) / GitHub Gist / local placeholder	public GitHub repo
Everything runs on CPU. How much compute time can I spend on the experiments?	quick ~15 min / moderate 45-90 min (recommended) / long 2-4 h	moderate

The agent recommended ConvNeXt V2 because it satisfies both halves of the assignment title at once: it is a modern **deep CNN** (a pure ConvNet with no attention) trained with a **deep (masked) autoencoder** objective; it is recent (CVPR 2023) and from a top venue; the paper contains a clear, small architectural idea (Global Response Normalization) that can be ablated; and its smallest variant (ConvNeXt V2-Atto, 3.7 M parameters) is feasible on a laptop CPU.

Later in the session I sent one more instruction — to delete the working folder and redo everything from a clean state ("給我一個全新的") — which the agent executed by re-creating all files and re-running all experiments; every number in this report comes from that second, clean run.

Reference. S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon and S. Xie, "ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders", CVPR 2023 (arXiv:2301.00808). Official code: github.com/facebookresearch/ConvNeXt-V2.

2. Algorithm description

2.1 Motivation

ConvNeXt (Liu et al., CVPR 2022) showed that a ResNet, once "modernised" with a patchify stem, 7×7 depth-wise convolutions, an inverted bottleneck, LayerNorm, GELU and fewer normalisation layers, matches Swin Transformers under supervised ImageNet training. In the meantime, vision Transformers had gained a second advantage: masked autoencoders (MAE, He et al. 2022) give them a very effective self-supervised pre-training. ConvNeXt V2 asks whether the same recipe can be transplanted to ConvNets. The paper finds that a naive "ConvNeXt + MAE" performs poorly, diagnoses the reason (feature collapse), and fixes it with a co-designed architecture change (GRN) and a convolution-specific masked autoencoder (FCMAE).

2.2 Fully Convolutional Masked AutoEncoder (FCMAE)

FCMAE is a masked autoencoder in which both encoder and decoder are convolutional (Figure 1b). Its components are:

- **Random masking.** The image is divided into patches at the resolution of the last encoder stage (32×32 pixels for 224×224 inputs) and 60 % of them are removed at random. The mask is up-sampled to the finer stages so every stage sees the same holes.
- **Encoder = ConvNeXt (V2).** A convolution kernel slides over masked and visible pixels alike, so the paper treats the visible pixels as a sparse point cloud and implements the encoder with sparse convolutions during pre-training; masked regions then cannot leak information. At fine-tuning time the sparse layers are converted back to ordinary dense convolutions without changing the weights. A dense equivalent, used in our implementation, is to zero the masked positions after every stage.
- **Lightweight decoder.** A single ConvNeXt block (512 channels in the paper) followed by a 1×1 prediction layer. Learnable mask tokens are inserted at masked positions before decoding. This asymmetry (heavy encoder, tiny decoder) is what makes MAE-style pre-training cheap.
- **Reconstruction target.** Mean-squared error between the prediction and the per-patch normalised pixel values, computed on masked patches only:

$$\mathcal{L} = \frac{1}{|M|} \sum_{p \in M} \left\| \hat{x}_p - \frac{x_p - \mu_p}{\sigma_p} \right\|_2^2 \quad (1)$$

where M is the set of masked patches and μ_p, σ_p are the mean and standard deviation of the pixels inside patch p .

2.3 The feature-collapse problem

Pre-training the original ConvNeXt V1 with FCMAE gives only a small gain over supervised training. Looking at the activations, the authors found many *dead or saturated channels* in the 4C-dimensional hidden layer of the inverted-bottleneck MLP: different channels were computing nearly the same thing. They quantify this with the average pairwise cosine distance between the channel feature maps $X_i \in \mathbb{R}^{H \times W}$:

$$d(X) = \frac{1}{C^2} \sum_{i=1}^C \sum_{j=1}^C \frac{1 - \cos(X_i, X_j)}{2} \quad (2)$$

A low value means redundant channels. ConvNeXt V1 + FCMAE shows a clear drop of d in the deeper layers, whereas MAE-trained ViTs and supervised ConvNeXts do not.

2.4 Global Response Normalization (GRN)

GRN is the fix. It is inspired by lateral inhibition in the visual cortex: it increases the *contrast between channels* so that they compete instead of collapsing onto the same feature. For an input $X \in \mathbb{R}^{H \times W \times C}$ it performs three steps with no extra hyper-parameters:

1. **Global feature aggregation:** spatially pool each channel with an L2 norm, $G(X)_i = \|X_i\|_2$, giving a vector in $\mathbb{R}^C$.
2. **Feature normalisation:** divisive normalisation of each channel by the aggregate over all channels,

$$\mathcal{N}(\|X_i\|) = \frac{\|X_i\|}{\sum_{j=1}^C \|X_j\|} \quad (3)$$

3. **Feature calibration:** rescale the input with the computed response, with a learnable affine transform and a residual path:

$$X_i \leftarrow \gamma \cdot X_i \mathcal{N}(\|X_i\|) + \beta + X_i \quad (4)$$

γ and β are initialised to zero so GRN is an identity at the start of training. GRN is inserted after GELU in every block, and the LayerScale parameter of ConvNeXt V1 becomes unnecessary and is removed (Figure 1a). The official implementation divides by the channel-mean of the norms instead of the sum; the difference is a constant factor absorbed by γ .

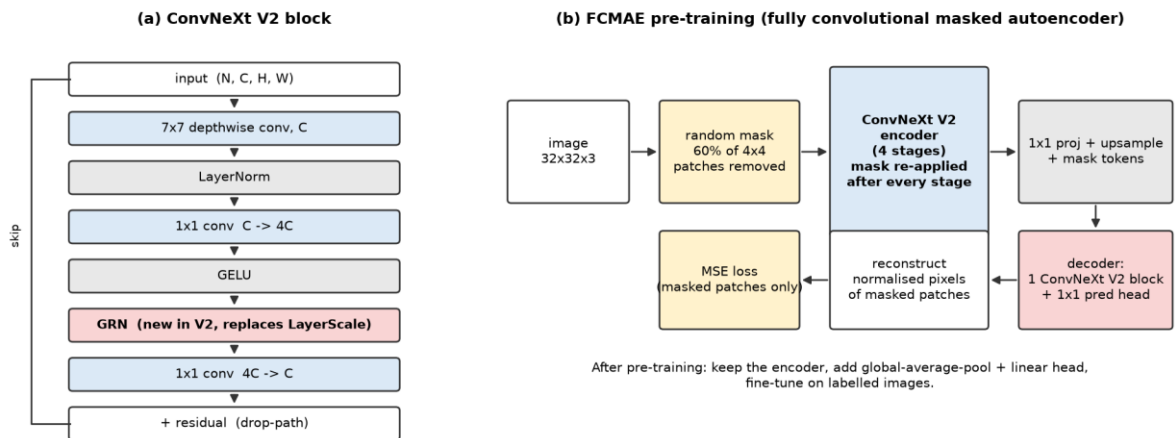


Figure 1. (a) The ConvNeXt V2 block — GRN (red) is the only difference from V1. (b) FCMAE pre-training pipeline as implemented in this assignment.

2.5 Results reported in the paper

With FCMAE pre-training and GRN, ConvNeXt V2 improves over V1 across the whole model family (from Atto, 3.7 M parameters, 76.7 % ImageNet-1K top-1, to Huge, 660 M parameters, 88.9 % with ImageNet-22K intermediate fine-tuning), and the two ingredients are shown to be complementary: GRN alone or FCMAE alone give little, while their combination gives the full gain (e.g. 83.7 $\rightarrow$ 84.6 % for the Base model). The pre-trained backbones transfer to COCO detection and ADE20K segmentation as well.

3. How the AI implemented the algorithm

3.1 Workflow of the agent

The implementation was done entirely by the agent inside the hw2/ folder in the following order (each step was announced in the chat before it was executed):

1. Environment check (Python, PyTorch, CPU count, GPU absence, GitHub authentication) and creation of the hw2/code, figures, results, data and report folders.
2. convnextv2.py: LayerNorm, GRN, ConvNeXt block (with a use_grn switch so the same file also builds the V1 block with LayerScale), DropPath, the 4-stage backbone and the Atto configuration (depths 2-2-6-2, widths 40-80-160-320).
3. fcmae.py: random mask generation, dense masked encoder, mask-token injection, single-block decoder, patchify/unpatchify and the normalised-pixel loss.
4. Smoke test with random tensors (shapes, exact mask ratio, one backward pass) and a speed benchmark; the channels_last memory format was found to be ~ 25 % faster on CPU and adopted.
5. data.py / prepare_data.py: CIFAR-10 pipeline. The official download ran at 70 kB/s (40 min), so the agent measured several mirrors, pulled the identical data as Parquet from the HuggingFace mirror at ~ 1 MB/s and repacked it into the torchvision batch format (two small bugs — pickle key type and torchvision's MD5 check — were caught by the agent and fixed).
6. pretrain.py, finetune.py, engine.py: AdamW + warm-up + cosine schedule, per-epoch JSON logging, checkpointing.
7. analysis.py: the cosine-distance collapse diagnostic of Section 2.3 (forward hooks on the MLP hidden layer of every block), 20-nearest-neighbour evaluation of the frozen encoder, reconstruction visualisation.
8. make_figures.py, run_all.py: figures and a one-command reproduction script (with a --quick smoke-test mode).
9. build_report.py: this report, generated with python-docx from the JSON results and converted to PDF with Word.

3.2 Key code produced by the agent

The GRN layer is eight lines and follows Eqs. (3)–(4) exactly (channels-last layout):

```

class GRN(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.gamma = nn.Parameter(torch.zeros(1, 1, 1, dim))
        self.beta = nn.Parameter(torch.zeros(1, 1, 1, dim))
        self.eps = eps

    def forward(self, x):
        # x: (N, H, W, C)
        gx = torch.norm(x, p=2, dim=(1, 2), keepdim=True) # step 1: aggregate
        nx = gx / (gx.mean(dim=-1, keepdim=True) + self.eps) # step 2: normalise
        return self.gamma * (x * nx) + self.beta + x # step 3: calibrate

```

The block simply drops LayerScale and inserts GRN after GELU:

```

x = self.dwconv(x) # 7x7 depth-wise conv
x = x.permute(0, 2, 3, 1) # -> channels last
x = self.norm(x) # LayerNorm
x = self.pwconv1(x); x = self.act(x) # 1x1 conv C->4C, GELU
if self.grn is not None: x = self.grn(x) # V2 only
x = self.pwconv2(x) # 1x1 conv 4C->C
if self.gamma is not None: x = self.gamma * x # V1 LayerScale only
return shortcut + self.drop_path(x.permute(0, 3, 1, 2))

```

Masking in the encoder (dense equivalent of the sparse convolutions) and the loss:

```

# ConvNeXtV2.forward_features
for i in range(4):
    x = self.downsample_layers[i](x)
    if mask is not None: x = x * (1 - resize(mask, x.shape[-2:]))
    x = self.stages[i](x)
    if mask is not None: x = x * (1 - resize(mask, x.shape[-2:]))

# FCMAE.forward_loss (norm_pix_loss)
target = self.patchify(imgs)
target = (target - target.mean(-1, keepdim=True)) / (target.var(-1, keepdim=True) + 1e-6)
** 0.5
loss = ((pred - target) ** 2).mean(-1) # per patch
loss = (loss * mask).sum() / mask.sum() # masked patches only

```

3.3 Adaptations for CIFAR-10 on a CPU

The paper works at 224×224 on GPUs; the agent made the following documented changes so that the whole study fits in about 1.5 hours on a 20-core laptop CPU:

- **Input resolution.** Stem = 2×2/stride-2 patchify conv (instead of 4×4/4), so the four stages run at 16×16, 8×8, 4×4 and 2×2 for 32×32 images.
- **Mask granularity.** The last stage is only 2×2, too coarse for masking, so the mask grid is decoupled from the encoder stride: 4×4-pixel patches (8×8 = 64 mask units per image), 60 % masked exactly as in the paper. The decoder up-samples the 2×2 encoder output to the 8×8 grid before injecting mask tokens.
- **Dense instead of sparse convolutions.** MinkowskiEngine is not available on Windows/CPU; the agent used the dense fallback that the official code also ships (zero the masked positions after every stage).
- **Model size.** ConvNeXt V2-Atto widths and depths; 3.386 M encoder parameters here (3.7 M in the paper — the difference is the ImageNet stem and 1000-class head). Decoder width 128 instead of 512.
- **Schedule.** 8 pre-training epochs and 30 fine-tuning epochs instead of 800 + 100/300. Runs were executed two or four at a time with 10 or 4 threads each, after measuring that CPU throughput did not improve beyond 10 threads per process.

4. Experiment settings and results

4.1 Settings

Table 1. Experimental settings.

Item	Setting
Dataset	CIFAR-10: 50,000 training images (used without labels for pre-training), a stratified 5,000-image labelled subset (500 per class) for supervised training, the full 10,000-image test set for evaluation
Backbone	ConvNeXt V2-Atto: depths (2, 2, 6, 2), widths (40, 80, 160, 320), stem $2 \times 2/2$, 3.386 M parameters (V1 variant: 3.373 M)
FCMAE pre-training	mask ratio 0.6 on 4×4 patches, decoder = 1 ConvNeXt block (128 ch), normalised-pixel MSE, AdamW ($\beta = 0.9/0.95$, wd 0.05), lr $1.5e-3$, 1 warm-up epoch + cosine, batch 128, 8 epochs, RandomResizedCrop(0.6–1) + horizontal flip
Supervised training	identical recipe for all four runs: AdamW (wd 0.05), lr $1e-3$, 3 warm-up epochs + cosine, batch 128, 30 epochs, label smoothing 0.1, drop-path 0.1, RandomCrop(pad 4) + horizontal flip
Hardware / software	20-core CPU, no GPU; Python 3.12, PyTorch 2.13 (channels_last); wall-clock: pre-training ≈ 51 min per model (two in parallel), fine-tuning ≈ 33 min per run (four in parallel)
Seeds	torch.manual_seed(0) for all runs; the 5k subset is drawn with numpy seed 0

4.2 Self-supervised pre-training

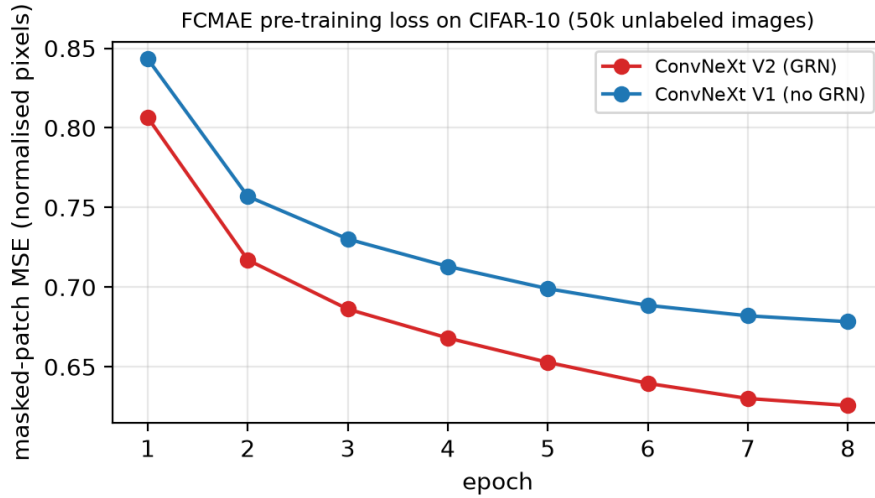


Figure 2. FCMAE reconstruction loss during pre-training for the V2 (GRN) and V1 (no GRN) encoders.

Table 2. Masked-patch MSE per epoch (lower is better).

Epoch	V2 (GRN) loss	V1 (no GRN) loss
1	0.8065	0.8433
2	0.7171	0.7570
3	0.6862	0.7302
4	0.6681	0.7130

5	0.6528	0.6990
6	0.6396	0.6886
7	0.6301	0.6820
8	0.6257	0.6783



Figure 3. Reconstructions by the pre-trained ConvNeXt V2 FCMAE on test images (rows: original, 60 % masked input, raw prediction, visible patches + prediction). Predicted patches are de-normalised with the ground-truth patch statistics for display.

Both encoders learn the inpainting task: the loss falls from about 0.806 to 0.626 (V2) and 0.678 (V1). Reconstructions (Figure 3) recover the coarse shape and colour of the objects even with only 40 % of the pixels visible, which is the expected behaviour of an MAE with a small decoder — it is trained to predict the mean of plausible completions, so results are blurry.

4.3 Feature collapse and the effect of GRN

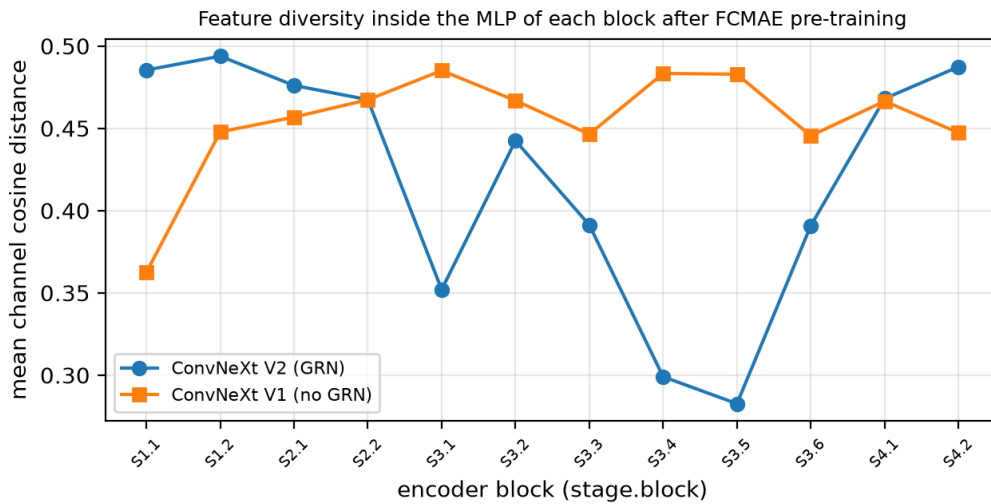


Figure 4. Mean channel cosine distance (Eq. 2) inside the MLP of every encoder block after FCMAE pre-training. Higher = more diverse channels.

Averaged over the 12 blocks the cosine distance is 0.420 for V2 with GRN and 0.455 for V1 without it. The V1 curve stays flat at roughly 0.45–0.49 from the second block onwards (a random, uncorrelated set of channels would give 0.5), so at this scale **the feature collapse reported in the paper does not occur**: there are no dead or duplicated channels for GRN to rescue. In V2 the GRN layer actually lowers the distance in stage 3 (blocks S3.1–S3.6: 0.360 vs 0.468), i.e. it makes the 4C hidden channels more correlated, before the distance recovers in stage 4. This is consistent with what GRN does mechanically — it amplifies channels with a large global response and suppresses the others — but it is the opposite of the paper's observation, which was made on a 3.7 M–660 M parameter model pre-trained for 800 epochs on 224×224 ImageNet. The most likely reading is that collapse is a phenomenon of long, large-scale MAE pre-training that an 8-epoch CIFAR-10 run simply does not reach. Interestingly GRN still helps the pretext task itself: the final reconstruction loss is 0.626 with GRN against 0.678 without (Table 2).

Before any labels are used, the quality of the frozen representation can be measured with a 20-NN classifier on the pooled encoder output: **38.36 %** (V2) versus **39.13 %** (V1) top-1 on the test set. Both are far above the 10 % chance level, so the self-supervised task does produce class-relevant features, and the two encoders are within a point of each other — a lower reconstruction loss did not translate into better frozen features here.

4.4 Transfer to classification with 5,000 labels

Table 3. CIFAR-10 test accuracy (10,000 images) after 30 epochs on 5,000 labelled images.

Block	Initialisation	Final test acc. (%)	Best test acc. (%)	Train time (min)
ConvNeXt V1 (no GRN)	random (scratch)	54.50	54.50	25
ConvNeXt V1 (no GRN)	FCMAE pre-trained	60.10	60.28	25
ConvNeXt V2 (GRN)	random (scratch)	55.81	55.81	33
ConvNeXt V2 (GRN)	FCMAE pre-trained	62.13	62.13	33

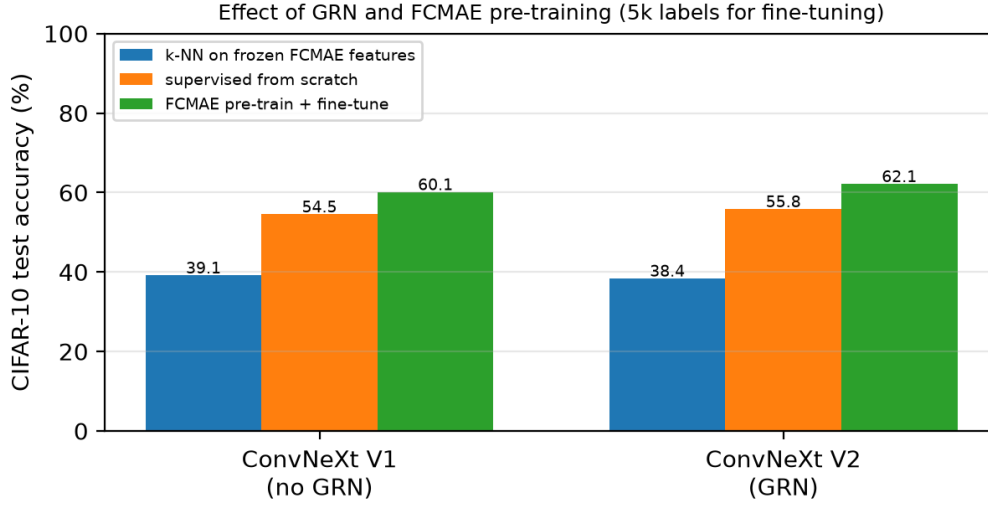


Figure 5. Summary of all evaluations: *k*-NN on frozen FCMAE features, supervised from scratch, FCMAE pre-training + fine-tuning.

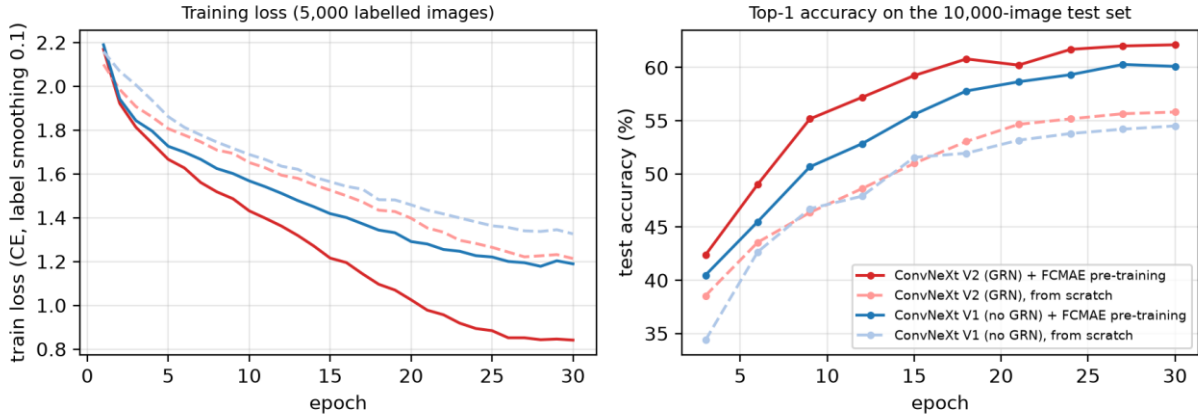


Figure 6. Training loss and test accuracy of the four supervised runs (solid = FCMAE-initialised, dashed = from scratch).

Observations. (i) FCMAE pre-training on unlabeled data is the dominant effect in this low-label regime: +6.32 points for the V2 block and +5.60 points for the V1 block, and the pre-trained models are ahead at every evaluation point of Figure 6 while also reaching a much lower training loss. (ii) GRN on its own, in supervised training from scratch, is worth only +1.31 points. (iii) Combined with FCMAE, GRN is worth +2.03 points, and the full ConvNeXt V2 recipe (GRN + FCMAE) is the best of the four configurations at 62.13 %. This is the same qualitative pattern as Table 3 of the paper — the architectural change matters most when paired with the self-supervised objective — even though Section 4.3 showed that the feature-collapse mechanism the paper uses to explain it is not visible at our scale. The absolute numbers are of course far below the paper's, and with a single seed and 30 epochs differences of about one point (such as the GRN-only gain) should be treated as indicative only.

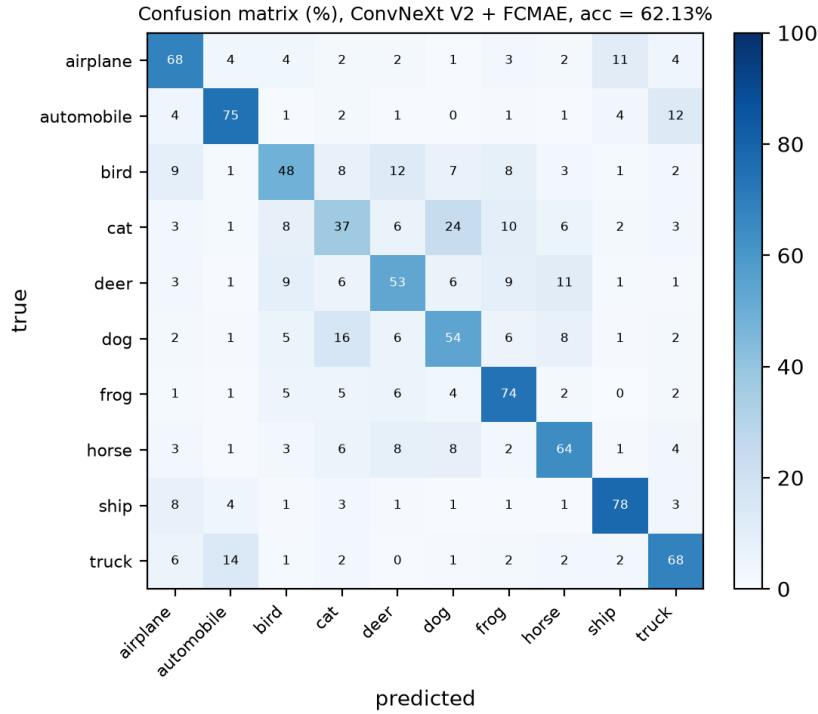


Figure 7. Confusion matrix of ConvNeXt V2 + FCMAE on the test set, rows normalised to 100 %.

Table 4. Per-class accuracy (%) of ConvNeXt V2 + FCMAE.

airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
68.2	74.8	48.5	37.0	53.4	54.2	74.5	64.4	78.1	68.2

The hardest classes are cat, bird, deer; the classic cat/dog and automobile/truck confusions dominate the off-diagonal of Figure 7.

4.5 Limitations

- Single seed per configuration; differences below about one percentage point should not be over-interpreted.
- 8 pre-training epochs are two orders of magnitude fewer than the paper's 800; FCMAE gains normally keep growing with longer pre-training.
- Dense masking instead of sparse convolutions slightly leaks information through zero-padding at patch borders.
- 32×32 images force a 2×2 last stage; the mask grid therefore had to be decoupled from the encoder stride.

5. What I have learnt from this AI assignment

- **About the algorithm.** I now understand why MAE pre-training does not transfer to ConvNets for free: convolutions mix masked and visible pixels (hence sparse convolutions / masking after every stage), and a ConvNeXt trained this way suffers from channel collapse in the wide MLP layer. GRN is a tiny, parameter-light normalisation that restores channel competition, and it is meant to be evaluated together with FCMAE rather than as a stand-alone block improvement. I also learnt that a paper's headline result and its explanation are

separate claims: the accuracy pattern (FCMAE + GRN best, GRN alone marginal) reproduced on CIFAR-10, but the collapse diagnostic did not — at small scale and short schedules there was no collapse to fix, so the mechanism cannot be confirmed from our data.

- **About working with an AI agent.** The most valuable prompts were not "write the code" but the decisions the agent asked me to make (which algorithm, how much compute, where to publish). Giving the agent access to the machine let it find and fix practical problems on its own — a 70 kB/s download replaced by a mirror, a 25 % speed-up from `channels_last`, a torchvision checksum that rejected repacked data. I also saw it make and correct mistakes (a syntax slip in a forward method, wrong pickle key types), which is a reminder that every generated line still has to be tested. Asking for a clean restart cost only the compute time: because everything was scripted, the agent rebuilt the folder and re-ran the study with one instruction.
- **About reproducible experiments.** Having the agent log every run to JSON and generate the figures and this report from those files means every number here can be regenerated with one command (`python run_all.py`). Honest reporting of the adaptations (dense masking, decoupled mask grid, short schedules) is as important as the numbers.
- **About the limits of the tool.** The agent cannot conjure compute: budgeting the CPU time (parallel processes, thread count, evaluation frequency) was a real part of the engineering, and the final accuracies are bounded by that budget rather than by the method.

6. Web page of the source code

All code, logs, figures and this report are publicly available at:

<https://github.com/HOIKAICHENG/CISC3024>

Repository layout: `code/` (all Python sources listed in Section 3.1), `results/` (JSON logs of every run and analysis), `figures/` (all figures of this report), `report/` (this document as DOCX and PDF).

Reproduce everything with `pip install -r requirements.txt && python code/run_all.py`.